

Fix: Settlement Window Race Condition (mojaloop/project#4375)

Problem

A race condition causes payer/payee settlement imbalance when settlement windows are closed while transfers are still being processed. Under load (e.g., 10 TPS), closing a settlement window produces a \$1 discrepancy: the payer's debit is captured but the payee's credit is missing.

Root Cause

Transfer processing spans two asynchronous stages connected by Kafka:

1. **Transfer fulfil handler** (central-ledger) writes `transferFulfilment` with `settlementWindowId` and produces a Kafka message to the position topic
2. **Position handler** (central-ledger) consumes that message LATER and writes `participantPositionChange` for the payee (COMMITTED state)

The `close()` function in central-settlement aggregates by INNER JOINing `transferFulfilment` -> `transferStateChange` -> `participantPositionChange`. If the position handler hasn't processed the payee's COMMIT yet, the INNER JOIN **silently excludes** the payee's entry, producing an imbalanced aggregation.

The existing retry mechanism (3 retries, 3s apart) never fires because `close()` succeeds with incomplete data -- it doesn't know the data is incomplete.

Solution

1. Pre-aggregation completeness check

File: `src/models/settlementWindow/facade.js` (lines 200-235)

Before running aggregation queries, count total transfers vs. transfers that have their `participantPositionChange` records. If counts don't match, throw an error that propagates up to the handler's retry loop.

Covers both regular transfers (`transferFulfilment`) and FX transfers (`fxTransferFulfilment`).

2. Post-aggregation balance validation

File: `src/models/settlementWindow/facade.js` (lines 270-285)

After inserting `settlementContentAggregation`, verify that amounts sum to zero per `settlementWindowContentId`. This is a safety net for edge cases where the completeness check passes but amounts still don't balance.

3. Increased retry defaults

File: `config/default.json` (lines 22-25)

Setting	Before	After
<code>RETRY_COUNT</code>	3	10
<code>RETRY_INTERVAL</code>	3000ms	2000ms
Total wait	9s	20s

Under load, the position handler may lag behind; this gives adequate time for it to catch up.

4. Enhanced error logging

File: `src/handlers/deferredSettlement/handler.js` (lines 110-112)

When retries are exhausted, the error message now includes:

- The specific settlement window ID
- That the window remains in PROCESSING state
- That manual intervention is required
- Context about the fulfil/position handler race condition

Files Modified

File	Lines Changed	Description
<code>src/models/settlementWindow/facade.js</code>	+55	Completeness check + balance validation
<code>config/default.json</code>	+2 / -2	Retry count/interval

File	Lines Changed	Description
<code>src/handlers/deferredSettlement/handler.js</code>	+3 / -1	Error message
<code>test/unit/models/settlementWindow/facade.test.js</code>	+144	2 new tests + updated success test

What Was NOT Changed

- **central-ledger**: No changes. The fix is at the point of consumption (central-settlement), not production.
- **Database schema**: No migrations. Uses existing tables and columns.
- **The `trx` issue in `savePayeeTransferResponse`** (central-ledger facade.js line 361): Low-priority secondary issue for a separate PR.

Testing

- All 764 existing unit tests pass
- New test: `close` throws when transfers have incomplete position changes
- New test: `close` throws when aggregation is imbalanced
- Updated: `close` success test stubs the new completeness/balance queries

Manual verification steps

See [Appendix A](#) for full procedures.

Appendix A: Manual Verification Procedures

Prerequisites

Infrastructure

- Kubernetes cluster with Mojaloop deployed (or local docker-compose environment)
- MySQL client with access to the `central_ledger` database
- Kafka CLI or UI tool (e.g., AKHQ, kafka-console-consumer) for monitoring topics
- Finance Portal UI accessible at the configured endpoint

Test Tools

- Apache JMeter (5.x+) with the Mojaloop JMeter test plan, or equivalent HTTP load generator
- Two configured DFSPs (e.g., `payerfsp`, `payeefsp`) with funded accounts
- `curl` or Postman for ad-hoc API calls

Environment Checks

Verify central-settlement is running the patched version

```
curl -s http://<central-settlement-host>:3007/health | jq .
```

Verify Kafka consumers are connected

```
kubectl logs deployment/central-settlement -n mojaloop | grep "Consumer connected"
```

Verify database connectivity

```
mysql -h <db-host> -u central_ledger -p central_ledger -e "SELECT 1"
```

Procedure 1: Reproduce the Race Condition (Pre-Fix Baseline)

Skip this procedure if testing only the patched version. This documents how to reproduce the original bug for comparison.

1. Open a fresh settlement window

Note the current open window ID

```
curl -s http://<central-settlement-host>:3007/v2/settlementWindows?state=OPEN | jq  
'.[0].settlementWindowId'
```

2. Start a sustained load test

JMeter: 10 TPS, peer-to-peer transfers between payerfsp and payeefsp

```
jmeter -n -t mojaloop-transfers.jmx \
```

```
-Jtps=10 \
```

```
-Jpayer=payerfsp \
```

```
-Jpayee=payeefsp \
```

```
-Jamount=1 \
```

-Jcurrency=USD \

-Jduration=120

3. While load is running, close the settlement window

- Via Finance Portal: Navigate to Settlement Windows > select the open window > Close
- Or via API:

WINDOW_ID=<current-open-window-id>

curl -X POST

http://<central-settlement-host>:3007/v2/settlementWindows/\${WINDOW_ID} \

-H "Content-Type: application/json" \

-d '{"state": "CLOSED", "reason": "Race condition test"}'

4. Check for imbalance

-- Compare payer debits vs payee credits for the closed window

SELECT

sca.settlementWindowContentId,

SUM(sca.amount) AS netAmount,

COUNT(*) AS entryCount

FROM settlementContentAggregation sca

JOIN settlementWindowContent swc

ON swc.settlementWindowContentId = sca.settlementWindowContentId

WHERE swc.settlementWindowId = <WINDOW_ID>

GROUP BY sca.settlementWindowContentId;

Pre-fix expected result: **netAmount** is non-zero (e.g., **1.0000** or **-1.0000**) for one or more content IDs, indicating a missed payee credit.

Procedure 2: Verify the Fix Under Load (Race Condition Path)

This is the primary verification: confirm that the completeness check triggers retries and the window eventually closes with balanced aggregation.

1. Deploy the patched central-settlement

If using Helm

```
helm upgrade central-settlement mojaloop/central-settlement \
```

```
--set image.tag=<patched-version> \
```

```
-n mojaloop
```

Verify the pod is running

```
kubectl rollout status deployment/central-settlement -n mojaloop
```

2. Open a Kafka consumer to monitor retry behavior

Watch the deferred settlement topic for close events

```
kafka-console-consumer \
```

```
--bootstrap-server <kafka-host>:9092 \
```

```
--topic topic-deferredsettlement-close \
```

```
--from-latest
```

3. Tail central-settlement logs in a separate terminal

```
kubectl logs -f deployment/central-settlement -n mojaloop | grep -E  
"(windowCloseRetry|pending position|imbalanced|done--C2)"
```

Expected log patterns during the race:

- `windowCloseRetry` -- retry triggered by completeness check failure
- `Window <ID> has N transfers with pending position changes` -- the new error message
- `done--C2` -- successful close after retries

4. Start the load test (same as Procedure 1, step 2)

```
jmeter -n -t mojaloop-transfers.jmx \
```

```
-Jtps=10 \
```

```
-Jpayer=payerfsp \
```

```
-Jpayee=payeefsp \
```

```
-Jamount=1 \
```

```
-Jcurrency=USD \
```

```
-Jduration=120
```

5. Close the settlement window ~30s after load starts

```
WINDOW_ID=$(curl -s  
http://<central-settlement-host>:3007/v2/settlementWindows?state=OPEN | jq -r  
'.[0].settlementWindowId')
```

```
echo "Closing window: ${WINDOW_ID}"
```

```
curl -X POST
```

```
http://<central-settlement-host>:3007/v2/settlementWindows/${WINDOW_ID} \
```

```
-H "Content-Type: application/json" \
```

```
-d "{\"state\": \"CLOSED\", \"reason\": \"Race condition fix verification\"}"
```

6. Verify retry behavior in logs

Wait up to 20s (the new retry window). You should see one or more `windowCloseRetry` entries followed by a `done--C2` success.

Pass criteria: The close eventually succeeds (no exhaustion error).

Fail criteria: You see `close failed after max retry count 10 has been exhausted in 20s` -- this means the position handler is too slow even for the increased timeout. Consider increasing `RETRY_COUNT` further or investigating position handler throughput.

7. Verify the window state

```
curl -s http://<central-settlement-host>:3007/v2/settlementWindows/${WINDOW_ID} | jq  
'state'
```

```
# Expected: "CLOSED"
```

8. Verify balanced aggregation

```
SELECT  
  
    sca.settlementWindowContentId,  
  
    SUM(sca.amount) AS netAmount,  
  
    COUNT(*) AS entryCount  
  
FROM settlementContentAggregation sca  
  
JOIN settlementWindowContent swc  
  
    ON swc.settlementWindowContentId = sca.settlementWindowContentId  
  
WHERE swc.settlementWindowId = <WINDOW_ID>  
  
GROUP BY sca.settlementWindowContentId;
```

Pass criteria: `netAmount` is `0.0000` for every `settlementWindowContentId`.

9. Verify transfer counts match

```
-- Total transfers in the window  
  
SELECT COUNT(DISTINCT tf.transferId) AS totalTransfers  
  
FROM transferFulfilment tf  
  
WHERE tf.settlementWindowId = <WINDOW_ID>;  
  
-- Transfers with position changes (should match above)  
  
SELECT COUNT(DISTINCT tf.transferId) AS completeTransfers  
  
FROM transferFulfilment tf  
  
JOIN transferStateChange tsc ON tsc.transferId = tf.transferId
```



```
JOIN participantPositionChange ppc ON ppc.transferStateChangeld =  
tsc.transferStateChangeld
```

```
WHERE tf.settlementWindowId = <WINDOW_ID>;
```

Pass criteria: Both counts are equal.

Procedure 3: Verify Normal Operation (No-Race Path)

Confirm the fix has no performance impact when there is no race condition.

1. **Stop any running load tests** (let all in-flight transfers complete)
2. **Wait 30s** for all Kafka consumers to drain
3. **Verify no pending position changes exist**

-- For the current open window, all transfers should be complete

```
SELECT
```

```
(SELECT COUNT(DISTINCT tf.transferId)
```

```
FROM transferFulfilment tf
```

```
WHERE tf.settlementWindowId = <OPEN_WINDOW_ID>) AS total,
```

```
(SELECT COUNT(DISTINCT tf.transferId)
```

```
FROM transferFulfilment tf
```

```
JOIN transferStateChange tsc ON tsc.transferId = tf.transferId
```

```
JOIN participantPositionChange ppc ON ppc.transferStateChangeld =  
tsc.transferStateChangeld
```

```
WHERE tf.settlementWindowId = <OPEN_WINDOW_ID>) AS complete;
```

Both values should be equal.

4. **Close the window and time the operation**

```
WINDOW_ID=$(curl -s  
http://<central-settlement-host>:3007/v2/settlementWindows?state=OPEN | jq -r  
'.[0].settlementWindowId')
```

```
echo "Closing window: ${WINDOW_ID}"
```

```
time curl -X POST  
http://<central-settlement-host>:3007/v2/settlementWindows/${WINDOW_ID} \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"state": "CLOSED", "reason": "Normal operation verification"}'
```

5. Check logs for retry activity

```
kubectl logs deployment/central-settlement -n mojaloop --since=30s | grep -c  
"windowCloseRetry"
```

```
# Expected: 0 (no retries needed)
```

6. Verify the window closed on first attempt

Pass criteria:

- No **windowCloseRetry** log entries
- Close completed in roughly the same time as before the fix (the 4 extra COUNT queries add negligible overhead)
- Window state is **CLOSED**
- Aggregation is balanced (same SQL as Procedure 2, step 8)

Procedure 4: Verify Retry Exhaustion Behavior

Confirm that when retries are genuinely exhausted, the error message is actionable and the window is left in a recoverable state.

1. Temporarily set very low retry limits (for testing only)

```
# Override via environment variable
```

```
kubectl set env deployment/central-settlement \
```

```
CSET_WINDOW_AGGREGATION__RETRY_COUNT=1 \
```

```
CSET_WINDOW_AGGREGATION__RETRY_INTERVAL=100 \
```

```
-n mojaloop
```

```
kubectl rollout status deployment/central-settlement -n mojaloop
```

2. **Start a high-throughput load test** (increase TPS to maximize race window)

```
jmeter -n -t mojaloop-transfers.jmx \
```

```
-Jtps=50 \
```

```
-Jpayer=payerfsp \
```

```
-Jpayee=payeefsp \
```

```
-Jamount=1 \
```

```
-Jcurrency=USD \
```

```
-Jduration=60
```

3. **Immediately close the window**

```
WINDOW_ID=$(curl -s  
http://<central-settlement-host>:3007/v2/settlementWindows?state=OPEN | jq -r  
'.[0].settlementWindowId')
```

```
curl -X POST  
http://<central-settlement-host>:3007/v2/settlementWindows/${WINDOW_ID} \
```

```
-H "Content-Type: application/json" \
```

```
-d "{\"state\": \"CLOSED\", \"reason\": \"Exhaustion test\"}"
```

4. **Check the error log**

```
kubectl logs deployment/central-settlement -n mojaloop --since=30s | grep "close failed"
```

Pass criteria: Error message contains:

- Settlement window <WINDOW_ID> close failed after max retry count 1 has been exhausted in 0.1s
- Window remains in PROCESSING state and requires manual intervention

- race condition between fulfil and position handlers

5. **Verify the window is in PROCESSING state** (not CLOSED, not corrupted)

```
curl -s http://<central-settlement-host>:3007/v2/settlementWindows/${WINDOW_ID} | jq  
'state'
```

```
# Expected: "PROCESSING"
```

6. **Verify the window can be retried manually** once transfers complete

```
# Wait for load to finish and position handler to drain
```

```
sleep 30
```

```
# Retry the close (the window is still in PROCESSING state)
```

```
# This requires re-sending the Kafka message or calling the domain service directly.
```

```
# The simplest approach is to restart central-settlement, which will re-process
```

```
# any uncommitted Kafka messages.
```

7. **Restore normal retry settings**

```
kubectl set env deployment/central-settlement \
```

```
  CSET_WINDOW_AGGREGATION__RETRY_COUNT=10 \
```

```
  CSET_WINDOW_AGGREGATION__RETRY_INTERVAL=2000 \
```

```
  -n mojaloop
```

```
kubectl rollout status deployment/central-settlement -n mojaloop
```

Procedure 5: FX Transfer Verification

If the environment supports FX transfers, verify the completeness check covers the FX path.

1. **Configure FX-capable DFSPs** with different source/target currencies (e.g., USD -> EUR)
2. **Run FX transfer load test**

```
jmeter -n -t mojaloop-fx-transfers.jmx \
```

```
-Jtps=10 \
```

```
-Jpayer=payerfsp \
```

```
-Jpayee=payeefsp \
```

```
-JsourceCurrency=USD \
```

```
-JtargetCurrency=EUR \
```

```
-Jduration=60
```

3. **Close the window mid-load** (same as Procedure 2, step 5)

4. **Verify FX completeness**

```
-- Total FX transfers in the window
```

```
SELECT COUNT(DISTINCT ftf.commitRequestId) AS totalFxTransfers
```

```
FROM fxTransferFulfilment ftf
```

```
WHERE ftf.settlementWindowId = <WINDOW_ID>;
```

```
-- FX transfers with position changes
```

```
SELECT COUNT(DISTINCT ftf.commitRequestId) AS completeFxTransfers
```

```
FROM fxTransferFulfilment ftf
```

```
JOIN fxTransferStateChange ftsc ON ftsc.commitRequestId = ftf.commitRequestId
```

```
JOIN participantPositionChange ppc ON ppc.fxTransferStateChangeId =  
ftsc.fxTransferStateChangeId
```

```
WHERE ftf.settlementWindowId = <WINDOW_ID>;
```

Pass criteria: Both counts are equal after the window closes successfully.

Results Summary Template

Procedure	Status	Notes
1. Reproduce race (baseline)	PASS / FAIL / SKIP	
2. Fix under load (race path)	PASS / FAIL	Retries observed: ____
3. Normal operation (no-race)	PASS / FAIL	Close time: ____ms
4. Retry exhaustion	PASS / FAIL	Error message correct: Y/N
5. FX transfer verification	PASS / FAIL / N/A	